01 QUESTION ONE · JAVA & OOPS

What is Java? Write concepts of OOPS which used in Java.

4 A4 PAGES
JAVA PROGRAM ON PAGE 4

WHAT IS JAVA?

Java is a high-level, object-oriented, robust and platform-independent programming language that is used to build a very wide range of software, from small desktop utilities to large enterprise systems. It was developed by **James Gosling** and first released by **Sun Microsystems in 1995**. While designing the language, Gosling wanted a language that could be written once and then run on any machine, and this idea became Java's famous motto — **"Write Once, Run Anywhere" (WORA)**. The project was originally named **"Oak"** after the tree outside Gosling's office, and it was later renamed Java after a popular brand of coffee.

The most important thing about Java is what happens after the code is compiled. Java source code is not compiled straight into machine language; instead the compiler produces a special file called **bytecode**, which can run on any operating system that has a Java environment installed. Because of this single feature, Java became one of the most widely used languages in the world. Today it runs banking systems, Android mobile apps, web applications, scientific software and embedded devices.

Main Features of Java

- **Simple:** the syntax is close to C and C++, but difficult features such as pointers, operator overloading and multiple inheritance were left out, which makes the language easier to learn.
- **Object-Oriented:** everything is organised around objects, and both the data and the methods that work on that data live together inside them.
- **Platform Independent:** the same bytecode runs on Windows, Linux or macOS — the basis of "Write Once, Run Anywhere".
- **Robust and Secure:** automatic memory management, exception handling and bytecode verification stop the program from crashing or being attacked easily.
- **Multithreaded:** several tasks can run at the same time inside one program, which makes it faster and more responsive.

Simple	Object-Oriented	Platform-Independent
Robust	Secure	Multithreaded
Portable	High Performance	Distributed

FIG. 1 — THE QUALITIES JAVA IS KNOWN FOR, SET OUT AS A GRID. THE THREE QUALITIES IN THE MIDDLE ROW ARE THE ONES A PROGRAMMER FEELS EVERY DAY: THE PROGRAM DOES NOT CRASH EASILY, THE BYTECODE IS VERIFIED BEFORE IT RUNS, AND SEVERAL TASKS CAN RUN TOGETHER.

JVM, JRE AND JDK

Before learning Java it is essential to understand three terms that actually make a Java program run. The **JVM (Java Virtual Machine)** is the engine that executes Java bytecode. Every operating system has its own JVM, and this is exactly why the same bytecode can run everywhere. The JVM also manages memory and performs **garbage collection**, which means automatically removing objects that are no longer needed.

The **JRE (Java Runtime Environment)** provides the JVM together with the libraries and files required to run a Java program. If you only need to run an existing program and not write a new one, installing the JRE is enough. The **JDK (Java Development Kit)** includes the JRE along with the tools needed to develop programs, such as `javac` (the compiler that turns source code into bytecode) and `java` (the launcher that runs the bytecode). In short, **JDK = JRE + Development Tools**.

How Platform Independence Works

When you write a program you create a source file (`.java`). When you compile it, the compiler converts it into a bytecode file (`.class`). That bytecode file is then executed by the JVM of whatever machine the program is running on — Windows, Linux or macOS — without any change to the code. The JVM acts as a translator standing between the bytecode and the real machine. Because the program never talks to the operating system directly, it stays independent of it, and this is the reason large organisations prefer Java for systems that must run on many different machines at once.

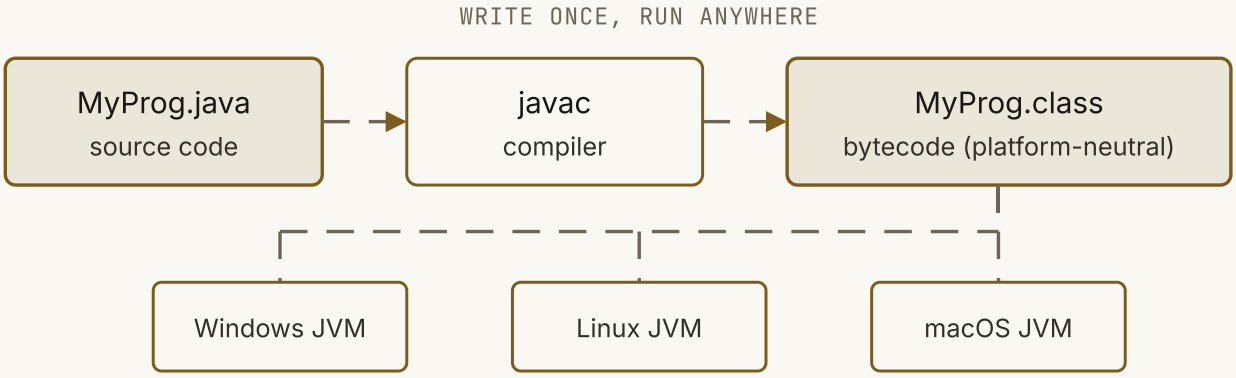


FIG. 2 — THE LIFE OF A JAVA PROGRAM. THE SAME **.CLASS** BYTECODE IS TAKEN BY THE JVM OF ANY OPERATING SYSTEM, SO THE SOURCE HAS TO BE WRITTEN ONLY ONCE.

Inside the JVM

The JVM itself is made of three main parts. The **class loader** finds the `.class` file and loads the bytecode into memory. The **memory areas** — the heap, where objects live, and the stack, where method calls are kept — hold the data while the program runs. The **execution engine** then runs the bytecode instruction by instruction and, where the same code runs again and again (a “hot spot”), it uses a **Just-In-Time (JIT) compiler** to convert that part into fast machine code. This is why a Java program may start a little slowly but then runs almost as fast as a program written directly in C. The same idea explains why Java is called “compiled and interpreted” rather than belonging to only one of the two groups.

CONCEPTS OF OOPS USED IN JAVA

Java is an **object-oriented programming language (OOPS)**, which means that problems are solved by organising the code into objects. OOPS has four main concepts, often called the four pillars — **Encapsulation, Inheritance, Polymorphism and Abstraction**. Each of them is explained below.

1. Encapsulation (Data Hiding)

Encapsulation means binding the data (variables) and the methods that work on that data into a single unit, which is a class, and then preventing the data from being changed directly from outside. A useful way to picture it is an ATM machine: all the internal work — checking the balance, dispensing cash — happens inside the machine, and the user can only use the buttons on the outside. In Java we declare the variables as `private` and provide `public` getter and setter methods to reach them.

2. Inheritance

Inheritance is the mechanism by which a new class acquires the properties and methods of an existing class. The existing class is called the **parent class** and the new one is called the **child class**. In Java the keyword `extends` is used. Its biggest advantage is **code reuse** — a piece of code written once does not have to be written again.

3. Polymorphism

Polymorphism literally means “many forms” — the ability of one name to behave in different ways. **Compile-time polymorphism** (method overloading) happens when a class has more than one method with the same name but different parameters. **Run-time polymorphism** (method overriding) happens when a child class provides its own version of a method already defined in the parent class.

4. Abstraction

Abstraction means showing only the essential details and hiding the complex internal working. A good example is driving a car — the driver needs only the steering, the brake and the accelerator, and not a knowledge of how the engine works. In Java, abstraction is implemented through **abstract classes** and **interfaces**, where only the declaration is written and the actual code is supplied by the child class.

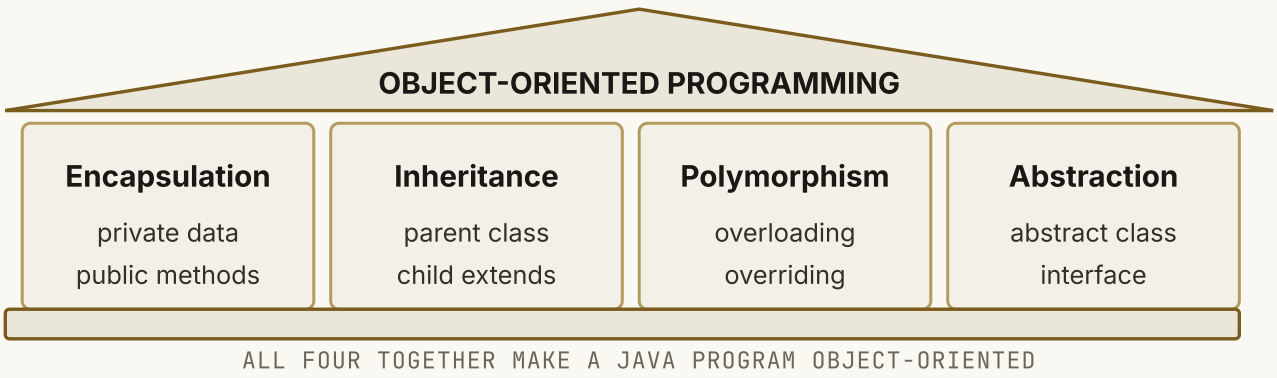


FIG. 3 — THE FOUR PILLARS, DRAWN AS FOUR PILLARS. EACH ONE SOLVES A DIFFERENT PROBLEM, AND A WELL-WRITTEN JAVA CLASS USUALLY SHOWS ALL FOUR AT WORK AT THE SAME TIME.

JAVA PROGRAM

THIS ONE PROGRAM DEMONSTRATES ALL FOUR OOP CONCEPTS TOGETHER. COPY IT ON PAGE 4.

DEMO.JAVA

```java
// Demo.java - all four OOP concepts in one program
abstract class Animal {                  // 1) ABSTRACTION
    String name;
    Animal(String name) { this.name = name; }
    abstract void sound();               // declared, no body
    void sleep() {                       // shared normal method
        System.out.println(name + " is sleeping");
    }
}
class Dog extends Animal {               // 2) INHERITANCE
    Dog(String name) { super(name); }
    @Override
    void sound() {                       // 3) POLYMORPHISM (overriding)
        System.out.println(name + " barks: Woof Woof");
    }
}
class Student {                          // 4) ENCAPSULATION
    private int marks;                   // data hiding
    Student(int marks) { this.marks = marks; }
    public int getMarks() { return marks; }
    public void setMarks(int m) {        // validated setter
        if (m ⩾ 0 && m ⩽ 100) marks = m;
        else System.out.println("Invalid marks");
    }
}
public class Demo {
    public static void main(String[] args) {
        Animal a = new Dog("Tommy");     // run-time polymorphism
        a.sound();
        a.sleep();
        Student s = new Student(85);
        s.setMarks(150);                 // rejected by validation
        System.out.println("Marks: " + s.getMarks());
    }
}
```

Output of the Program

OUTPUT

```
Tommy barks: Woof Woof
Tommy is sleeping
Invalid marks
Marks: 85
```

CONCLUSION

Java is a platform-independent, object-oriented language in which the abstract class gave us abstraction, **extends** gave us inheritance, **@Override** showed polymorphism and the private members gave us encapsulation. These four pillars are the foundation of Java and of all object-oriented programming.

02 QUESTION TWO · ENCAPSULATION

Write one example for encapsulation and define each function.

4 A4 PAGES
PROGRAM ON PAGE 3, FUNCTIONS ON PAGE 4

WHAT IS ENCAPSULATION?

Encapsulation is one of the pillars of OOPS in which we bind the **data** (variables) and the **functions** (methods) that operate on that data together into a single unit — that is, into a class — and then prevent the data from being reached directly from outside. A simple way to remember it is to think of a capsule of medicine: the medicine is sealed inside the capsule and cannot be touched from the outside. This is exactly why the technique is called encapsulation.

Encapsulation is needed because if the data of an object stays open, anyone can change it in a wrong way and errors begin to appear in the program. For example, if the *balance* of a bank account were public, a careless line of code could set it to a negative value. Through encapsulation we make the data **private**, and to reach it we provide **public** methods known as getters and setters. These methods can check the value before allowing it. This controlled access is called **Data Hiding** — the data stays hidden, yet safe access is always available.

Advantages of Encapsulation

- **Data Security:** since the data is private, it cannot be corrupted directly from outside the class.
- **Controlled Access:** the data can be reached only through getters and setters, where validation can be performed.
- **Flexibility:** the internal code can be changed later without disturbing the code that uses the class.
- **Reusability and Easy Testing:** a fully encapsulated class is self-contained, so it is easy to reuse and easy to test.
- **Maintenance:** a single owner of the data means the program is easier to understand and to keep correct.

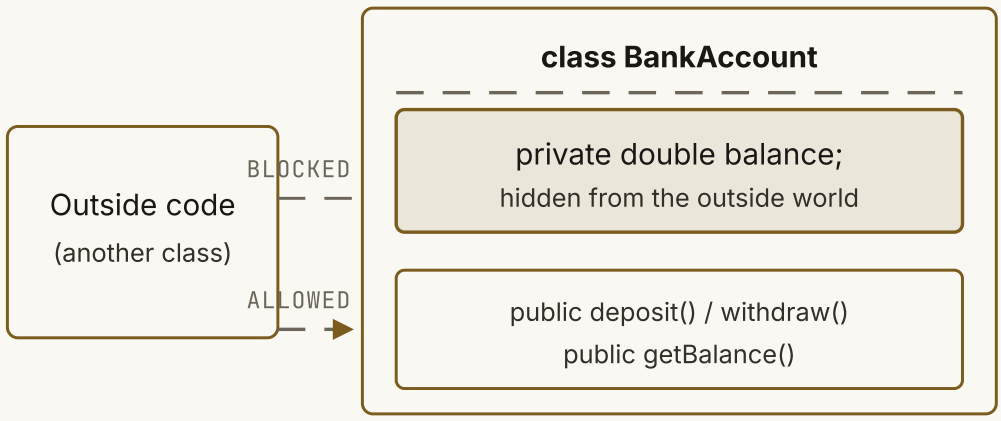


FIG. 1 — ENCAPSULATION. THE DATA STAYS PRIVATE AND THE OUTSIDE CODE MAY REACH IT ONLY THROUGH THE PUBLIC METHODS, WHERE THE VALUES CAN BE CHECKED FIRST.

EXAMPLE — THE BANKACCOUNT CLASS

To see encapsulation in practice we take a real-life example, a **BankAccount**. In this class the account's **balance** is important and sensitive data. If anyone could change the balance directly, the whole banking system could be damaged. Therefore we make the balance **private**, and to reach it we provide public methods such as getBalance, setBalance, deposit and withdraw.

Inside this class there are six functions in all. The purpose of each one is listed in the table below; the program is written on Page 3 and a full definition of every function is given on Page 4.

MEMBER / FUNCTION	TYPE	PURPOSE
`private String owner`	Variable	Name of the account holder (hidden)
`private double balance`	Variable	Current balance (hidden)
`BankAccount(...)`	Constructor	Sets the name and opening balance
`getOwner()`	Getter	Reads the private name
`getBalance()`	Getter	Reads the private balance
`deposit(amount)`	Method	Adds money after checking it is positive
`withdraw(amount)`	Method	Removes money after checking the balance
`setBalance(newB)`	Setter	Updates the balance only if it is valid

Notice that the two data members are private while every function that touches them is public. That is encapsulation in one line — private data, public methods.

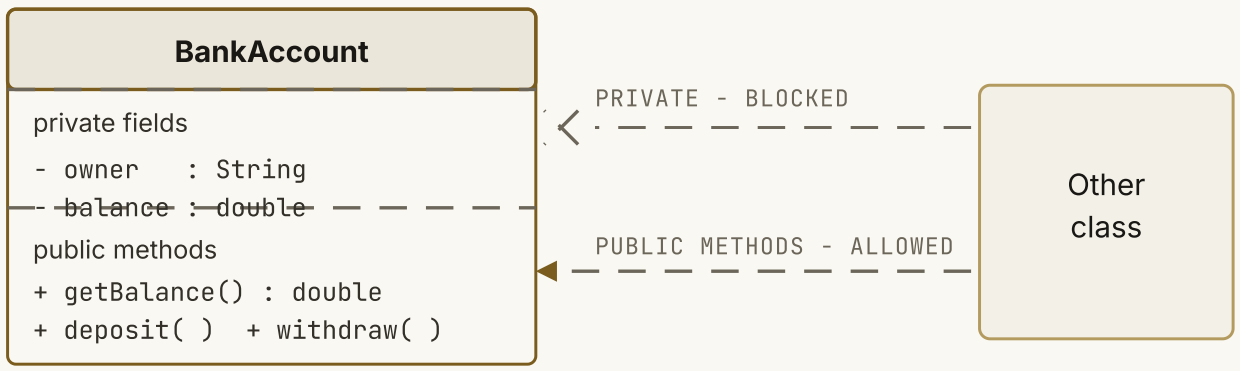


FIG. 2 — THE SHAPE OF AN ENCAPSULATED CLASS. THE FIELDS SIT BEHIND A WALL; THE ONLY DOORS IN THAT WALL ARE THE PUBLIC METHODS, AND EVERY VISITOR — THAT IS, EVERY OTHER CLASS — MUST ENTER THROUGH THEM.

JAVA PROGRAM

AN EXAMPLE OF ENCAPSULATION USING A BANK ACCOUNT. THE BALANCE IS PRIVATE AND CAN BE REACHED ONLY THROUGH PUBLIC METHODS.

BANKACCOUNT.JAVA

```java
// BankAccount.java - encapsulation in action
class BankAccount {
    private String owner;               // hidden data
    private double balance;             // hidden data

    public BankAccount(String owner, double balance) {   // constructor
        this.owner = owner;
        this.balance = balance;
    }
    public String getOwner() { return owner; }        // getter
    public double getBalance() { return balance; }    // getter

    public void deposit(double amount) {              // adds money
        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Deposited: Rs." + amount);
        }
    }
    public void withdraw(double amount) {             // removes money
        if (amount > 0 && amount ⩽ balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: Rs." + amount);
        } else {
            System.out.println("Error: insufficient balance");
        }
    }
    public void setBalance(double newBalance) {       // validated setter
        if (newBalance ⩾ 0) balance = newBalance;
        else System.out.println("Error: balance cannot be negative");
    }
}
public class Main {
    public static void main(String[] args) {          // testing
        BankAccount acc = new BankAccount("Rahul Kumar", 5000);
        acc.deposit(2500);         // 5000 + 2500 = 7500
        acc.withdraw(1000);        // 7500 - 1000 = 6500
        acc.setBalance(-500);      // rejected by validation
        System.out.println("Owner   : " + acc.getOwner());
        System.out.println("Balance : Rs." + acc.getBalance());
    }
}
```

Definition of Each Function

1. Constructor — BankAccount(String owner, double balance): a constructor is a special function that runs automatically when an object is created. Its name is the same as the class and it has no return type. Its job is to set the `owner` and the opening `balance` as soon as an account is created. Because it lies inside the class, it can reach the private members directly.

2. getOwner(): this is a **getter** (accessor). Its only job is to return the value of the private variable `owner`. It changes nothing — it only reads — and since the variable is private, the name can be read from outside only through this public method.

3. getBalance(): this is also a **getter**. It returns the current value of the private variable `balance`. As the balance is never shown openly, this method provides a safe way to read it.

4. deposit(double amount): this method adds money to the account. It first checks that `amount` is positive, and only then adds it to the balance. Because it lies inside the class, it can update the private balance directly.

5. withdraw(double amount): this method removes money. It performs two checks — that the amount is positive, and that it is not greater than the current balance. If both pass, the amount is subtracted; otherwise the message "insufficient balance" is printed.

6. setBalance(double newBalance): this is a **setter** (mutator). It updates the private balance, but only after checking that the new value is not negative. This validation is the main benefit of encapsulation — because the data is not open, a wrong value is caught before it is stored.

Output of the Program

OUTPUT

```
Deposited: Rs.2500.0
Withdrawn: Rs.1000.0
Error: balance cannot be negative
Owner   : Rahul Kumar
Balance : Rs.6500.0
```

CONCLUSION

This example shows how encapsulation keeps the balance completely safe. Nobody from outside can set it to a negative value, because the setter validates it first. That is the real benefit of encapsulation — **data hiding, controlled access and validation** working together.

03 QUESTION THREE · CLASS ACCESS

What is the difference between private and public class?

4 A4 PAGES
TABLE + PROGRAM ON PAGE 4

WHAT IS A CLASS AND WHAT IS AN ACCESS MODIFIER?

In Java a **class** is a blueprint from which objects are created. A class contains data (variables) and functions (methods), and every object built from it follows the same design. For example, the class `Student` describes what every student has — a roll number, a name and marks — and each real student is an object of that class.

When a class is declared, it may be given an **access modifier**. An access modifier decides **from where** that class can be used. Java provides four of them — `public`, `protected`, `default` (written with no keyword at all) and `private`. When we speak about the access modifier of a class itself, the two that matter most in practice are the **public class** and the **private class**, and the difference between them decides how far the class will reach.

Why the Difference Matters

If a class is made too open, other parts of the program can depend on it and its internal working becomes difficult to change. If it is made too closed, the rest of the program cannot use it at all. Choosing the right access is therefore a balance between **availability** and **hiding**, and that balance is exactly what the public/private decision controls. The two are explained in detail on the following pages, and compared with a table and a program on Page 4.

A useful way to think about it is to compare a class with a room. A **public class** is like a room whose door opens onto the street, so anyone may walk in; such a room must therefore be kept tidy, because whatever lies inside it becomes the business of the whole town. A **private class** is like an inner room that can be reached only from the house itself; the owner may keep anything in it, in any order, because no stranger ever sees it. Good programmers keep most of their classes as the second kind of room and open only a small, carefully chosen set of doors to the outside world. This habit is what makes a large program manageable, and it is why access modifiers are among the first things a Java programmer must learn to use wisely.

It must also be remembered that the access of a class and the access of its members are two separate matters. A class may be public while its variables are private, and this is in fact the usual arrangement — the class itself is open for use, but the data inside it stays hidden. This combination is exactly what encapsulation asks for, and it is why the two ideas are always taught together: the modifier on the class decides who may use the class, and the modifiers on the members decide who may touch the data inside it.

PUBLIC CLASS

When the keyword `public` is written before a class, that class becomes accessible **anywhere in the whole project**, even from a completely different package. For example, if we write `public class Student`, then any other class — in our own package or in another package, after importing it — can create objects of `Student` and use its public methods.

There is one important rule that goes with a public class: **the name of the .java file must be exactly the same as the name of the public class.** If the file is named `Student.java`, then it may contain only one public class, and that class must be `Student`. If a second public class is written in the same file, the compiler reports an error. This rule exists so that the compiler and the JVM can always find the class by its file name.

A public class usually contains the `main()` method of a program, because the Java launcher looks for `main()` inside a class that can be reached from outside. In short, a public class is the **face** that the program shows to the world.

Characteristics of a Public Class

- **Accessible everywhere:** it can be used from any package and any class.
- **File name must match:** the name of the public class must match the name of the .java file.
- **Used to build interfaces:** when we want others to use our class, we make it public.
- **One per file:** a single .java file may contain only one public class.

Other Access Modifiers for Reference

Between the two extremes of public and private lie two more modifiers. A **default** (package-private) class, written with no keyword at all, is visible only inside its own package; this is the most common choice for helper classes that must not be exposed to the world. A **protected** class is visible inside its package and also to its subclasses in other packages, so it is used where inheritance must reach across packages. A private top-level class is not allowed at all — only a nested class may be private — which is why the choice for a top-level class is really between public and default. Knowing this ladder — public, then default, then private — is what lets a programmer decide exactly how far a class should be allowed to travel.

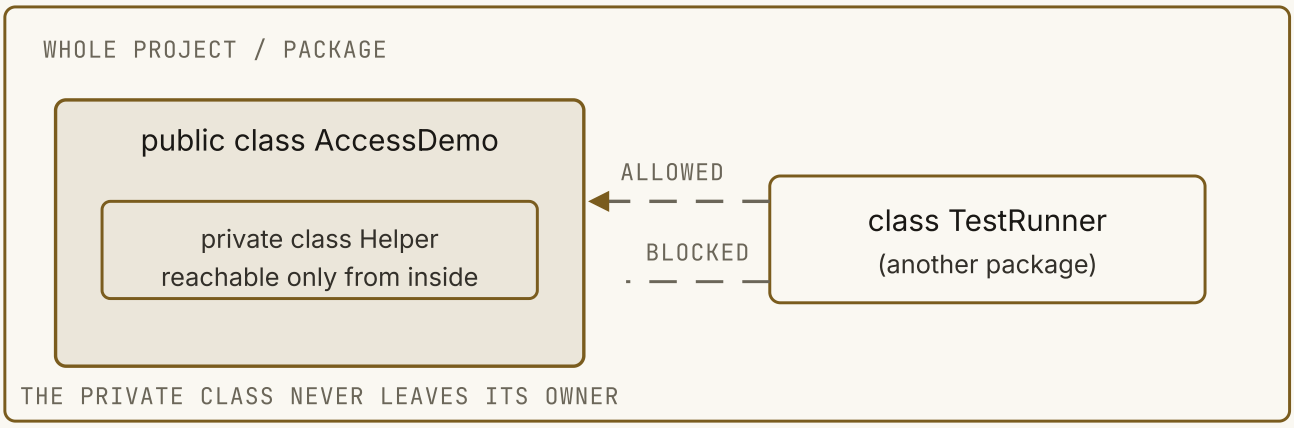


FIG. 1 — HOW FAR EACH CLASS CAN TRAVEL. A PUBLIC CLASS IS OPEN TO THE WHOLE PROJECT; A PRIVATE CLASS IS TRAPPED INSIDE THE CLASS THAT OWNS IT.

PRIVATE CLASS

When the keyword `private` is written before a class, that class becomes accessible **only inside the class in which it is defined**, and not from outside at all. In Java such a class works as a **nested class** (also called an inner class). This means that a private class can be used only inside its **enclosing (outer) class**, and no class outside it can create an object of it.

We use a private class when we want the entire implementation of some part of the program to stay hidden from the outside world. For example, if a class `Outer` contains a `private class Helper`, then `Helper` can be used only inside `Outer`. Nothing outside can even name it, let alone create it. The outside world sees only the public face of `Outer`, while all the helper work goes on privately behind it.

There is one more rule that must be remembered: a **top-level class** (a class written at the very top of a file) **can never be private**; it may only be `public` or default. **Only nested classes can be private.** This is a fundamental rule of Java, and it is the reason a private class is always an inner class.

Characteristics of a Private Class

- **Accessible only in the enclosing class:** no class outside can reach it directly.
- **Implementation stays hidden:** the outside world sees only the interface, not the internal work.
- **Can only be a nested class:** a top-level class can never be private.
- **Improves encapsulation:** helper code is kept completely out of sight.

A private class is useful in a very common real situation. Suppose a class must do a small internal calculation — say, format an amount into words, or check a password against a set of rules — and this work is of no interest to anyone else. If the helper class were public, any part of the program could call it, and the helper would slowly become part of the program's official surface, which would make the class hard to change later. By keeping it private, the author is free to rewrite or even remove the helper whenever he wishes, because nothing outside depends on it. This is exactly the principle behind the phrase **“hide what can be hidden”**: the smaller the public face of a class, the easier the class is to use, to test and to improve.

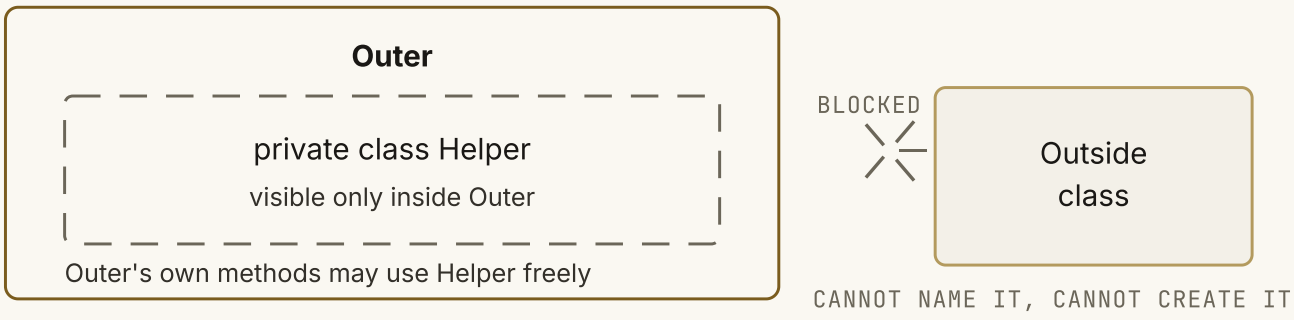


FIG. 2 — A PRIVATE CLASS INSIDE ITS ENCLOSING CLASS. THE ENCLOSING CLASS KEEPS THE KEY, ITS OWN METHODS WALK STRAIGHT IN, AND EVERY CLASS OUTSIDE THE WALL IS STOPPED AT THE DOOR.

Difference between Private and Public Class

BASIS	PUBLIC CLASS	PRIVATE CLASS
Accessibility	Anywhere in the whole project	Only inside the enclosing class
Purpose	Used when the class must be used from outside	Used when the implementation must stay hidden
Where it is written	Can be top-level or nested	Can only be a nested (inner) class
File-name rule	Its name must match the .java file name	No such rule applies
Encapsulation	Less — the class is open	More — the class is hidden
Example	`public class Student`	`private class Helper`

JAVA PROGRAM

ACCESSDEMO.JAVA

```java
// AccessDemo.java - public class vs private (nested) class
public class AccessDemo {                  // PUBLIC class
    private class Helper {                 // PRIVATE nested class
        void show() {
            System.out.println("Helper works only inside AccessDemo");
        }
    }
    public void startWork() {              // public method
        Helper h = new Helper();           // allowed: we are inside
        h.show();
    }
}
class TestRunner {                         // default (non-public) class
    void tryHelper() {
        // AccessDemo.Helper h = new AccessDemo.Helper();  // ERROR here
        System.out.println("Helper cannot be used from here");
    }
    public static void main(String[] args) {
        AccessDemo demo = new AccessDemo();
        demo.startWork();
        new TestRunner().tryHelper();
    }
}
```

Output of the Program

OUTPUT

```
Helper works only inside AccessDemo
Helper cannot be used from here
```

CONCLUSION

A public class is accessible throughout the project and is used to provide an interface, while a private class is accessible only inside its enclosing class and is used to hide the implementation. A private class can only be nested, and a public class must match its file name. That is the real difference between the two.